REPORT :

Screenshots:

1A	./test_objects output showing all tests passing

```
tasyashetty@Ubuntu24:~/Desktop/PES1UG24AM411-pes-vcs$ make test_objects
make: 'test_objects' is up to date.
tasyashetty@Ubuntu24:~/Desktop/PES1UG24AM411-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9
db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227e
d0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
tasyashetty@Ubuntu24:~/Desktop/PES1UG24AM411-pes-vcs$
```

1B	find .pes/objects -type f showing sharded directory structure

```
tasyashetty@Ubuntu24:~/Desktop/PES1UG24AM411-pes-vcs$ find .pes/objects
-type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11e
c4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e
566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9
aafc12
tasyashetty@Ubuntu24:~/Desktop/PES1UG24AM411-pes-vcs$
```

2A	./test_tree output showing all tests passing

```
tasyashetty@Ubuntu24:~/Desktop/PES1UG24AM411-pes-vcs$ make test_tree
./test_tree
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

2B	xxd of a raw tree object (first 20 lines)

```
tasyashetty@Ubuntu24:~/Desktop/PES1UG24AM411-pes-vcs$ find .pes/objects -type f

.pes/objects/bd/bc125c112a63700951eee0c77c02b4ad9637c3ca74bab27473e3d856bf2e80
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
tasyashetty@Ubuntu24:~/Desktop/PES1UG24AM411-pes-vcs$ xxd .pes/objects/bd/bc125c112a637009
51eee0c77c02b4ad9637c3ca74bab27473e3d856bf2e80
 | head -20
00000000: 636f 6d6d 6974 2031 3830 0074 7265 6520  commit 180.tree
00000010: 3665 6631 3962 3431 3232 3563 3533 3639  6ef19b41225c5369
00000020: 6631 6331 3034 6434 3564 3864 3835 6566  f1c104d45d8d85ef
00000030: 6139 6230 3537 6235 3362 3134 6234 6239  a9b057b53b14b4b9
00000040: 6239 3339 6464 3734 6465 6363 3533 3231  b939dd74decc5321
00000050: 0a61 7574 686f 7220 5461 7379 6120 5368  .author Tasya Sh
00000060: 6574 7479 203c 5045 5331 5547 3234 414d  etty <PES1UG24AM
00000070: 3431 313e 2031 3737 3633 3539 3832 320a  411> 1776359822.
00000080: 636f 6d6d 6974 7465 7220 5461 7379 6120  committer Tasya
00000090: 5368 6574 7479 203c 5045 5331 5547 3234  Shetty <PES1UG24
000000a0: 414d 3431 313e 2031 3737 3633 3539 3832  AM411> 177635982
000000b0: 320a 0a46 6972 7374 2063 6f6d 6d69 74    2..First commit
bash: syntax error near unexpected token `|'
```

3A	pes init → pes add → pes status sequence

```
tasyashetty@Ubuntu24:~/Desktop/PES1UG24AM411-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
tasyashetty@Ubuntu24:~/Desktop/PES1UG24AM411-pes-vcs$ ./pes add file1.txt file2.txt
tasyashetty@Ubuntu24:~/Desktop/PES1UG24AM411-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  test_sequence.sh
  untracked:  Makefile
  untracked:  commit.h
  untracked:  .gitignore
  untracked:  test_objects
  untracked:  commit.c
  untracked:  pes.c
  untracked:  test_tree.c
  untracked:  index.h
  untracked:  index.c
  untracked:  pes.h
  untracked:  test_objects.c
  untracked:  README.md
  untracked:  object.c
  untracked:  tree.c
  untracked:  tree.h
  untracked:  test_tree
```

3B	cat .pes/index showing the text-format index

```
tasyashetty@Ubuntu24:~/Desktop/PES1UG24AM411-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776359997 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776360002 6 file2.txt
```

4A	pes log output with three commits

```
tasyashetty@Ubuntu24:~/Desktop/PES1UG24AM411-pes-vcs$ ./pes log
commit a63274d93b15310dc91cd1358f2f73e89350593fc862da07c25c39292dc8fcc0
Author: Tasya Shetty <PES1UG24AM411>
Date:   1776360134

    Add farewell

commit 6d83cdbec5e00a8afa7221d7cb7ec6a9c54ad5985fc3c2084431c2b25f4c7447
Author: Tasya Shetty <PES1UG24AM411>
Date:   1776360134

    Add world

Add farewell

commit 9daf4551b56cf902c4f245630c28005f0b837f015a6b183387958fe9e429a343
Author: Tasya Shetty <PES1UG24AM411>
Date:   1776360134

    Initial commit farewell

commit bdbc125c112a63700951eee0c77c02b4ad9637c3ca74bab27473e3d856bf2e80
Author: Tasya Shetty <PES1UG24AM411>
Date:   1776359822

    First commitirstQ
```

4B	find .pes -type f | sort showing object growth

```
tasyashetty@Ubuntu24:~/Desktop/PES1UG24AM411-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6d/83cdbec5e00a8afa7221d7cb7ec6a9c54ad5985fc3c2084431c2b25f4c7447
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
.pes/objects/9d/af4551b56cf902c4f245630c28005f0b837f015a6b183387958fe9e429a343
.pes/objects/a6/3274d93b15310dc91cd1358f2f73e89350593fc862da07c25c39292dc8fcc0
.pes/objects/bd/bc125c112a63700951eee0c77c02b4ad9637c3ca74bab27473e3d856bf2e80
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

Full integration test (make test-integration)

```
tasyashetty@Ubuntu24:~/Desktop/PES1UG24AM411-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: bf99dd9d27ea... Initial commit

Log after first commit:
commit bf99dd9d27eabfb195b3eb73ceb59f8c810d3ac1a7be13d953c13ee1199af59d
Author: Tasya Shetty <PES1UG24AM411>
Date:   1776363003

    Initial commit


--- Second Commit ---
Committed: 2bceb1645eee... Update file.txt
```

```
committed: 889b9537801fa... Add farewell

--- Full History ---
commit 889b9537801fa1317b2dd355210cca95e152532b3c3efa6ac97e582e08d3b9a5
Author: Tasya Shetty <PES1UG24AM411>
Date:   1776363003

    Add farewell

commit 2bceb1645eeef3e1daeda3d0a91d937b82f482728825a2bebcca5f57a1832ce3
Author: Tasya Shetty <PES1UG24AM411>
Date:   1776363003

    Update file.txt

commit bf99dd9d27eabfb195b3eb73ceb59f8c810d3ac1a7be13d953c13ee1199af59d
Author: Tasya Shetty <PES1UG24AM411>
Date:   1776363003

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
889b9537801fa1317b2dd355210cca95e152532b3c3efa6ac97e582e08d3b9a5

--- Object Store ---
Objects created:
8
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/2b/ceb1645eeef3e1daeda3d0a91d937b82f482728825a2bebcca5f57a1832ce3
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
.pes/objects/88/9b9537801fa1317b2dd355210cca95e152532b3c3efa6ac97e582e08d3b9a5
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/bf/99dd9d27eabfb195b3eb73ceb59f8c810d3ac1a7be13d953c13ee1199af59d
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
```

Analysis Questions:

Q5.1: Implementing `pes checkout <branch>`

When a user switches branches, the system has to carefully update several parts of the repository.

Inside the .pes directory, three main things change. The HEAD file is updated to point to the new branch (or directly to a commit in case of a detached HEAD). The branch reference file must exist and contain the correct commit hash. The index file is completely rewritten to match the new branch's snapshot.

In the working directory, files are updated to match the target branch. Some files may be replaced, some removed, and some left unchanged. Untracked files are not touched. New files from the target branch are created, and files that no longer exist in that branch are deleted.

This process is complex for several reasons. First, safety is critical. If a file has uncommitted changes and would be overwritten, the checkout must stop to prevent data loss. Second, the operation should be atomic, meaning it either completes fully or not at all. Third, efficiency matters because large repositories cannot afford to recheck every file fully. Instead, metadata like file size and timestamps are used. Finally, permissions and symlinks must be handled correctly across systems.

Q5.2: Detecting a Dirty Working Directory

To check whether the working directory has uncommitted changes, the system uses the index as a reference.

It starts by reading the current HEAD commit and loading the index. For each tracked file, it compares the file's metadata (like size and modification time) with what's stored in the index. If they differ, the file is hashed to confirm whether it was actually modified.

If a file is missing, it is considered deleted. Then, the target branch's tree is loaded.

If a file has been modified in the working directory and is also different in the target branch, a conflict is detected and checkout is stopped. Otherwise, the file can be safely updated.

This method works efficiently because it avoids hashing every file and only verifies files that appear to have changed.

Q5.3: Detached HEAD and Commit Recovery

When working in a detached HEAD state, commits are still created normally and stored in the object database. However, they are not linked to any branch.

Once the user switches to another branch, these commits lose their reference and can eventually be removed during garbage collection.

To recover such commits, users can check the reflog (if available), inspect logs manually, or search the object store. After finding the commit hash, they can create a new branch pointing to it and continue working.

To avoid this situation, it is best to create a branch before making changes.

Q6.1: Garbage Collection (Mark-and-Sweep)

Garbage collection removes objects that are no longer reachable.

In the mark phase, the system starts from all branch and tag references and recursively marks every reachable object (commits, trees, blobs).

In the sweep phase, it scans all stored objects and deletes those that were not marked.

A hash set is used to track reachable objects efficiently. For large repositories, most objects are typically still reachable, while a smaller portion is removed.

Q6.2: Race Condition During Garbage Collection

A dangerous situation can occur if garbage collection runs at the same time as a commit.

If GC starts and marks all current objects, and then a new commit is created afterward, the new objects may not be marked. If GC deletes them before the commit is fully linked via a branch reference, the repository becomes corrupted.

To prevent this, systems like Git use locking mechanisms, atomic updates, and reflogs to ensure that new objects are not accidentally deleted during GC.

These protections ensure consistency even when multiple operations happen at the same time.